

Engscope

An Engineer's Life

10.2 I2C Basic Functions

I2C Basic Functions

In this section, I will outline some of the basic functions for the I2C module on a PIC24. Since the bulk of the electrical functions were explained in [section 1 of the tutorial](#), this section mainly focuses on the I2C module of the PIC24.

Word of Caution: Silicon Errata

There is a bug in the PIC24FJ64GA004 series of chips (16GA002, 32GA002, 48GA002, 64GA002, 16GA004, 32GA004, 48GA004, 64GA004). The bug is listed in the silicon errata section on [Microchip's website](#). Go there and search for Silicon Errata, and select the chip you are using. If the chip you are using is not affected by the error then you are good to go. However, if the error is present you'll need to try to get around the problem. In the PIC24F series of chips, there are two I2C modules. The I2C1 module does NOT work correctly in the master mode. You can read more about it in my [post on Microchip's developer forums](#).

There is a work around to making the I2C1 module work, but it is clumsy and does not always guarantee success. For this reason, if you are thinking about using the I2C module, I would suggest using only the I2C2 module on the PIC24F64 family, or just avoid the whole PIC24FJ64 family all together. I much prefer the PIC24H family anyways.

The Registers

As always, the starting place to understanding how to a module is with the registers. In this case, there are mainly 3 registers of great concern.

TABLE 3-8: I2C1 REGISTER MAP

SFR Name	SFR Addr	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
I2C1RCV	0200	—	—	—	—	—	—	—	—	Receive Register							
I2C1TRN	0202	—	—	—	—	—	—	—	—	Transmit Register							
I2C1BRG	0204	—	—	—	—	—	—	—	—	BaudRate Generator Register							
I2C1CON	0206	I2CEN	—	I2CSOL	SOLREL	IPMIEN	A10M	DISELW	SMEN	GCEN	STREN	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
I2C1STAT	0208	ACKSTAT	TRSTAT	—	—	—	SOL	GCSTAT	ADD10	IMCOL	I2CDIV	D_A	P	S	R_W	RBF	TBF
I2C1ADD	020A	—	—	—	—	—	—	Address Register									
I2C1MSK	020C	—	—	—	—	—	—	Address Mask Register									

The first is I2CxCON. This register is the main control register. With it, you can initiate start bit, stop bits, and test to see if acknowledge bits were received. The next one is the I2CxSTAT. This register allows you to see what the current status of the I2C module. This may include polling to see if there are errors, bit collisions, receive buffer overflows and other occurrences. Lastly there is the I2CxBRG, which determines the bit rate at which the module operates.

There are other minor registers such as I2CxRCV, I2CxTRN and I2CxMSK. The I2CxRCV register is used to retrieve received data. The I2CxTRN register is used to send data. The I2CxMSK is not used in our example since it is only used when the module is set to Slave mode.

When using the I2CxCON register, you must be careful with some of the initiation bits. When setting a Start or Stop bit, the hardware will automatically clear the set bit when it is ready to proceed with processing data. This means that it is not enough to just initiate a start bit and then proceed to send or receive data. You must keep polling that bit until the PIC automatically clears the bit, and only then proceed with processing data. There will be more on this matter later in the basic functions.

Using the I2C Module

The way I approach the I2C module is analogous to making sentences. I start with letters, which adds up to words, and combine them to make sentences. The individual base commands and the fiddling with the registers are kind of like the “letters”. I use these commands to form basic functions, functions that write or read one byte (words), and finally I combine these functions into full packets, with address, sub-address, and data (full sentences).

Recall that a basic “write” packet is composed of a chip (or slave) address, a sub-address, and the data. A “read” packet is composed of a chip address and a sub-address in the write configuration, followed by a chip address and the data sent by the slave chip.

Each of the individual bits and bytes must be sent in a specific order. The whole packet must be sent correctly for the slave to understand the intention of the master.

There are quite a few functions in this tutorial. I don’t think I need to go through every one of them, but I will annotate here and there if things are not very clear.

Initiation

Several things must be initiated before using the module. I use the following function.

“void i2c_init(int BRG)”

```
//function initiates I2C1 module to baud rate BRG
void i2c_init(int BRG)
{
    int temp;

    // I2C1BRG = 194 for 10Mhz OSCI with PPL with 100kHz I2C clock
    I2C1BRG = BRG;
    I2C1CONbits.I2CEN = 0;        // Disable I2C Mode
    I2C1CONbits.DISSLW = 1;       // Disable slew rate control
    IFS1bits.MI2C1IF = 0;         // Clear Interrupt
    I2C1CONbits.I2CEN = 1;        // Enable I2C Mode
    temp = I2CRCV;                // read buffer to clear buffer full
    reset_i2c_bus();              // set bus to idle
}
```

The “reset_i2c_bus()” function is a basic function that set the I2C bus to an idle state. This way the bus is ready to be used immediately after initiation.

Usually, slave devices are rated for up to 100 kHz. The “fast” specifications require that devices function up to 400 kHz. In the newest spec for the I2C bus, the “ultra fast” allows devices to function up to 1 Mhz, which is really fast. I rarely need anything that fast, and generally just stick to something slow like 40 kHz or so. It doesn’t really matter what baud rate you pick since the controller is sending the clock signal anyways. With a 10 MHz oscillator, I usually set the BRG value to about 100. This works for most applications.

Basic Functions

In order to initiate a start bit or a stop bit, there are some intricacies that a user must be aware. First, as I mentioned earlier, the hardware automatically clears certain bits in the I2CxCON register. Again, I cannot emphasize how important it is to read and UNDERSTAND the datasheet. Because of the way these bits work, when I want to initiate a start or stop condition, I must write a loop that keeps on polling the bit until the hardware clear, after which I can proceed to do my sending and receiving. Another way to do this would be to put a long delay, long enough that assure the occurrence of the hardware clear. I usually take the former approach, as it requires more coding, but is more efficient. The following is how I write the function to initiate a start bit and restart bit:

“void i2c_start(void)”



```
//function initiates a start condition on bus
void i2c_start(void)
{
    int x = 0;
    I2C1CONbits.ACKDT = 0;          //Reset any previous Ack
    DelayuSec(10);
    I2C1CONbits.SEN = 1; //Initiate Start condition
    Nop();

    //the hardware will automatically clear Start Bit
    //wait for automatic clear before proceeding
    while (I2C1CONbits.SEN)
    {
```

```
        DelayuSec(1);  
        x++;  
        if (x > 20)  
            break;  
    }  
    DelayuSec(2);  
}
```

“void i2c_restart(void)”

```
void i2c_restart(void)  
{  
    int x = 0;  
  
    I2C1CONbits.RSEN = 1;        //Initiate restart condition  
    Nop();  
  
    //the hardware will automatically clear restart bit  
    //wait for automatic clear before proceeding  
    while (I2C1CONbits.RSEN)  
    {  
        DelayuSec(1);  
        x++;  
        if (x > 20)        break;  
    }  
  
    DelayuSec(2);  
}
```

I also need a function to initiate a stop bit. However, I realize that when I initiate a stop bit, what I really want to

do is put the I2C bus in an idle state. For this reason, I reset a whole bunch of registers every time I initiate a stop bit to clear them of any previous errors and ACKs. Again beware of automatic hardware clears as in the case of I2C1CONbits.PEN.

“void reset_i2c_bus(void)”

```
//Resets the I2C bus to Idle
void reset_i2c_bus(void)
{
    int x = 0;

    //initiate stop bit
    I2C1CONbits.PEN = 1;

    //wait for hardware clear of stop bit
    while (I2C1CONbits.PEN)
    {
        DelayuSec(1);
        x ++;
        if (x > 20) break;
    }
    I2C1CONbits.RCEN = 0;
    IFS1bits.MI2C1IF = 0; // Clear Interrupt
    I2C1STATbits.IWCOL = 0;
    I2C1STATbits.BCL = 0;
    DelayuSec(10);
}
```

Send and Receive, Write and Read

The basic “send” function is written as follows:

“char send_i2c_byte(int data)”

```
//basic I2C byte send
char send_i2c_byte(int data)
{
    int i;

    while (I2C1STATbits.TBF) { }
    IFS1bits.MI2C1IF = 0; // Clear Interrupt
    I2CTRN = data; // load the outgoing data byte

    // wait for transmission
    for (i=0; i<500; i++)
    {
        if (!I2C1STATbits.TRSTAT) break;
        DelayuSec(1);

    }
    if (i == 500) {
        return(1);
    }

    // Check for NO_ACK from slave, abort if not found
    if (I2C1STATbits.ACKSTAT == 1)
    {
        reset_i2c_bus();
        return(1);
    }
}
```



```
    DelayuSec(2);  
    return(0);  
}
```

The return of the function sends back a 1 if there is an error, and a 0 if the transmission went through correctly.

I use two different read functions. This is because a sequential read requires the use of an ACK sent by the master to initiate the next byte. A random read of a single byte of data does not require an ACK to be sent.

“char i2c_read(void)”

```
//function reads data, returns the read data, no ack  
char i2c_read(void)  
{  
    int i = 0;  
    char data = 0;  
  
    //set I2C module to receive  
    I2C1CONbits.RCEN = 1;  
  
    //if no response, break  
    while (!I2C1STATbits.RBF)  
    {  
        i ++;  
        if (i > 2000) break;  
    }  
  
    //get data from I2CRCV register  
    data = I2CRCV;
```

```
    //return data  
    return data;  
}
```

“char i2c_read_ack(void)”

```
//function reads data, returns the read data, with ack  
char i2c_read_ack(void) //does not reset bus!!!  
{  
    int i = 0;  
    char data = 0;  
  
    //set I2C module to receive  
    I2C1CONbits.RCEN = 1;  
  
    //if no response, break  
    while (!I2C1STATbits.RBF)  
    {  
        i++;  
        if (i > 2000) break;  
    }  
  
    //get data from I2CRCV register  
    data = I2CRCV;  
  
    //set ACK to high  
    I2C1CONbits.ACKEN = 1;  
  
    //wait before exiting
```

```

    DelayuSec(10);

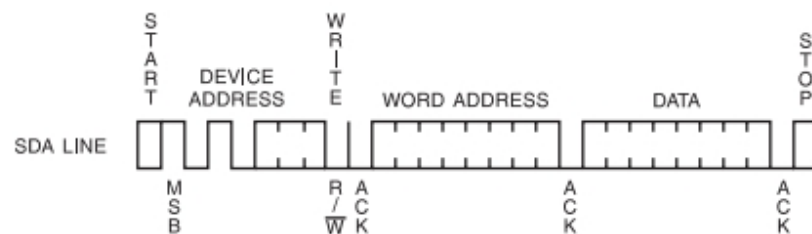
    //return data
    return data;
}

```

Putting it Together

There you have it. Those are the basic “words” in a sentence. To use my basic I2C functions, I need to follow the I2C standard.

The I2C standard requires that a random write be sent in the following manner:



The PIC (master) must send a start bit, followed by a device address, with the WRITE command. The slave then sends an ACK to acknowledge the byte. Next the master sends the sub-address, followed by a slave ACK. Lastly it sends the data, followed by a slave ACK, and finishes with a stop bit.

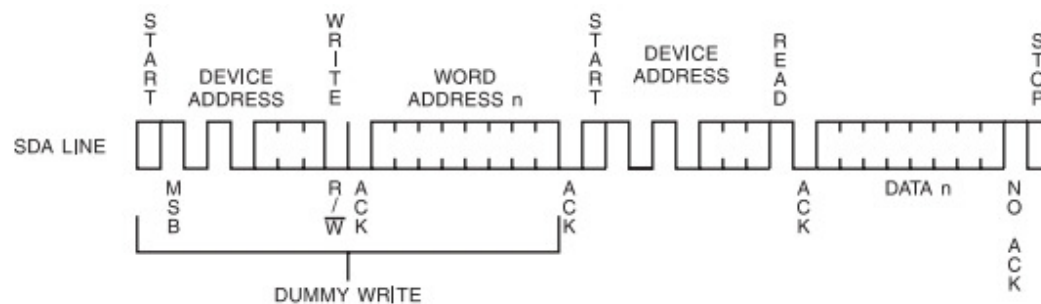
If you look closely at the “send_i2c_byte(int data)” function, you’ll see that it waits for an ACK from the slave. If a /ACK is detected (no acknowledge detected), then it returns an error and sets the I2C bus back to an IDLE state. This is exactly what we need.

A random write function is composed of “words” of basic functions in the exact manner required by the I2C specifications:

“void I2Cwrite(char addr, char subaddr, char value)”

```
void I2Cwrite(char addr, char subaddr, char value)
{
    i2c_start();
    send_i2c_byte(addr);
    send_i2c_byte(subaddr);
    send_i2c_byte(value);
    reset_i2c_bus();
}
```

In the same way, a random read must be sent in the following manner:



The random read function starts off exactly the same way as a random write. However, after the 2nd byte, the master must initiate a RESTART bit, followed by the device address with a read command. The slave sends and ACK, followed by the data requested by the master. The master then sends a /ACK, before issuing a stop bit. I

wrote the following function to emulate the I2C requirements:

“char I2Cread(char addr, char subaddr)

```
char I2Cread(char addr, char subaddr)
{
    char temp;

    i2c_start();
    send_i2c_byte(addr);
    send_i2c_byte(subaddr);
    DelayuSec(10);

    i2c_restart();
    send_i2c_byte(addr | 0x01);
    temp = i2c_read();

    reset_i2c_bus();
    return temp;
}
```

The function returns the data received.

Polling

Polling is an important function during I2C operations. I usually use a poll function before any read or writes because I want to make sure that the device I think is on my I2C bus is actually on my I2C bus. I eliminates communication errors that otherwise might escape the regular functions' error handling abilities. The premise of

the polling function is to wait for the ACK from a slave device. If a slave device sends back an ACK, it means that it is on the I2C bus and functioning properly.

“unsigned char I2Cpoll(char addr)”

```
unsigned char I2Cpoll(char addr)
{
    unsigned char temp = 0;

    i2c_start();
    temp = send_i2c_byte(addr);
    reset_i2c_bus();

    return temp;
}
```

There you have it. That’s how I2C works on an PIC24 I2C module. The last section of the I2C tutorial will deal with usage, examples, and advanced functions.

[Table of Contents](#)



96 Responses to 10.2 I2C Basic Functions



Ridha says:

November 28, 2011 at 9:34 am

I will try your suggestion. However, the problem is that sometimes when the system stops working, the PIC24 doesn't receive the interrupt signal (INT) from the device (MCP23008). Another difficulty is that this problem occurs randomly, sometimes after after several hours. I am new in this and I have been stuck with this problem for several weeks now. I have tried putting capacitors on the interrupt and serial communication lines and I have changed the pull-up resistors from 10K to 2.2K to no avail. Could the fact that I am using PLL have anything to do with it? Is my baud rate (157) correct? Do you think that this problem will be resolved if I use to SPI (with MCP23S08) instead of I2C? Any other ideas to help me are greatly appreciated.



jliu83 says:

November 28, 2011 at 11:43 am

@Ridha. Run the program in debug mode and wait for the problem to occur. Then stop the debugger to see where in your code it is stuck. Perhaps you will find something wrong with the code. I prefer SPI to I2C, but this is a personal preference. SPI is easier to debug, and must faster than I2C.

-J



Ridha says:

December 1, 2011 at 9:08 am

Hi J. Thanks for your prompt replies. I tried your suggestion of turning the I2C module off when not in use and turn it on when an interrupt is received, but unfortunately it did not work. I run the program in debug mode. When the problem occurs, the code doesn't get stuck. It is just that no interrupt from the device (MCP23008) is received; therefore, the keypad

does not respond and the program keeps running. It looks like the device is completely disabled for some crazy unknown reason, and the problem is that it is unpredictable when the problem occurs.



jliu83 says:

December 1, 2011 at 8:38 pm

@Ridha. Sounds like the problem is in your MCP23008. I'm not familiar with the device, but I don't see the point of using it. If you just need more GPIOs, just go with the next larger package. This chip might have been useful perhaps 10 years ago, but these days, the solutions to not enough IOs is probably to change the microcontroller.

-J



Ridha says:

December 1, 2011 at 11:51 pm

It is too late for me to change the microcontroller at this stage. I am supposed to release the schematics for the PCB for the project next week, and the whole design is complete, except for this keypad interface part. Besides, Microchip have a demo board (GPIODM-KPLCD) which uses the same device in the same way, except with a PIC18F4550 controller which works fine. So it cannot be the device. Perhaps my keypad is the cause of the problem?? I am really running out of solutions. My last one is to use the MCP23S08 device, which is the brother of MCP23008, but which uses SPI instead of I2C. To do that I have to learn SPI and modify the code...



jliu83 says:

December 2, 2011 at 7:15 am

@Ridha. SPI isn't so hard. Besides you will have learned something very useful for future projects. I would give it a shot,

regardless of whether you can get your circuit working or not. Failed circuits are stepping stones to working ones, as long as you figure out why they failed (eventually).



Ridha *says:*

December 2, 2011 at 8:31 am

Thank you for the positive words and advice. I really appreciate it.



Stefano *says:*

December 2, 2011 at 10:52 am

Thanks for your tutorials, they were helpful!

I was wondering if you were so kind to help me sorting out a problem when reading from an EEPROM.

I have a PIC24FJ64GA004 and a 24LC16B low density eeprom (8 256-bytes pages) and this is the code I use to read a single byte from an address in the first page:

```
unsigned char leggiByteEEPROM(unsigned char indirizzo){
unsigned char output;
IdleI2C2();
StartI2C2();
IdleI2C2(); //Wait to complete

if (MasterWriteI2C2( 0xA0 )== -1) {}; // control byte
while(1) flasha; // error write collision
}
IdleI2C2(); //Wait to complete
```

```
if (MasterWriteI2C2( indirizzo )== -1) {;
ledon;
while(1); // error write collision
}
Delay(10);
IdleI2C2(); //Wait to complete

//RestartI2C2(); //Send the Restart condition <----- NO CLOCK! ----
IdleI2C2(); //Wait to complete

if (MasterWriteI2C2( 0xA1 )== -1) {; //transmit read command
ledon;
while(1); // error write collision
}
while(I2C2STATbits.TRSTAT); // wait till bus free
IdleI2C2(); //Wait to complete

I2C2CONbits.RCEN = 1; // enable rc
while(!I2C2STATbits.RBF); // wait till rcv buff full
I2C2STATbits.I2COV = 0;
output=I2C2RCV;
IdleI2C2();

I2C2CONbits.ACKDT = 1; //Set for NotACK
I2C2CONbits.ACKEN = 1;
while(I2C2CONbits.ACKEN); //wait for ACK to complete
I2C2CONbits.ACKDT = 0; //Set for NotACK

StopI2C2(); //Send the Stop condition
IdleI2C2(); //Wait to complete
return(output);
```

```
}
```

The line marked with NO CLOCK means that if i uncomment that line (as it is supposed to be), when i set the RCEN flag the master (pic) doesn't generate the clock on SCL so the slave (eeprom) doesn't send the byte and the code hangs indefinitely waiting for I2C2STATbits.RBF to be set.

Leaving that line commented means that the SCL clock is generated (as it is supposed to be) after setting RCEN, but obviously the master receives bogus data (0xFF, as the SDA line stays high).

I've uploaded two pictures of my oscilloscope with the Restart line uncommented (1st picture) and commented (2nd) and you can clearly see that scl clock isn't generated as it is supposed to be. The first half is a writing sequence and after a small delay starts a reading sequence. In the 2nd picture the SCL and SDA lines stay low. <http://imgur.com/a/5fNkA>

I hope you can help!



jliu83 says:

December 5, 2011 at 8:13 am

@Stefano. Your IdleI2C2() is probably looping indefinitely, although the code in the Microchip Peripheral library looks good, I have seen buggy restart in I2C modules. Try issuing a Stop, then a Start instead of a restart, and see if it works. Let me know what happens.

-J



Stefano says:

December 5, 2011 at 8:14 am

Solved, it seems a problem related to slew rate control (that i should have disabled).

**Ridha says:**

December 11, 2011 at 1:54 pm

Hi J, Regarding my problem I described above, I have tried SPI, but it also failed the same way. i.e. After a lapse of time the keypad stops responding. When this happens, there is no activity on the SCK line when a key is pressed. Do you think that the keypad switch bouncing can cause this to happen?

**jliu83 says:**

December 12, 2011 at 11:22 am

@Ridha. How long is the “no activity” duration before this error state occurs? I do not think it is the keypad bounce, but rather something more fundamental such as the power, or an error in the way the chip is used. Is the power supply to the chip quiet? What happens to the power supply after this “no activity” duration?

-J

**Ridha says:**

December 12, 2011 at 4:08 pm

J, The “non activity” duration is variable: it could be 15 minutes, 1 hour, 2 hours or more. Its is really unpredictable when this error state occurs, and that is a problem. Right now I am taking the 3.3V and Ground to power my circuit containing the device from the Explorer16 development board, which is itself powered by a 9V DC adapter. I don't think at all that power is the issue. The chip is used the same way the demo board GPIODM-KPLCD. The system is simple: it has 3 main components: the MCU, the GPIO device and the the leypad. The only difference with the demo board (which works fine) is the keypad and the MCU (PIC24 instead of PIC18), and of course the software which is adapted from the demo board software. What could make the PIC24 stop outputing the clock (SCK) signal? Thanks.



jliu83 says:

December 13, 2011 at 8:30 am

@Ridha. Sounds very strange to me. Without seeing the actual demo in action I'm afraid there's not much I can help you with. One final thought is perhaps a detection mechanism to reboot the device whenever it senses an unresponsive device. You can poll the SPI device, and if there's no response (due to no SCK output), then reboot the device, which should reset the whole circuit. Perhaps this is a workaround that might work in your situation.

-J



Ridha says:

December 13, 2011 at 9:30 am

J, thank you very much for all your support. I will probably pursue the reboot option you suggested. I will update you if/when I solve this problem. Have a good day!



Daniel says:

March 22, 2012 at 8:02 am

Congratz, dude.

Is it correct? You show the function `send_i2c_byte` twice!



jliu83 says:

March 27, 2012 at 2:15 pm

@Daniel. Yeah, I think when I changed hosting, I must have copied something wrong. Will get fixed very soon.

-J



Mohan says:

June 14, 2012 at 5:07 am

I2C1CONbits.ACKEN = 1; in receive with acknowledgement is not working for me with pic24fj64ga006 it shows no stop while bus is free in proteus simulator. why?



jliu83 says:

June 14, 2012 at 11:26 am

@Mohan. I don't work with simulators, it just add another layer of possible error. This may include bad interface, bad models inaccurate constants etc. Spend \$25 and build a real circuit. You'll save yourself a lot of headache.

-J



syahrin says:

July 9, 2012 at 12:53 am

Hi,

I really like to try this code.

btw what about the restart i2c_restart(void) code? is it the same as reset_i2c_bus(void)?



jliu83 says:

July 9, 2012 at 10:24 am

@syahrim. Restart is not the same as reset. Please read over the I2C bus protocol for further info. Restart is used to initiate a second start signal (for example, during a read, pull down sda). A reset is used before and after a packet is sent through the bus (release both sda and scl).



kishore says:

July 28, 2012 at 10:26 am

Hi Friends,

I am using PIC24F16KA102 controller to get data through I2C bus. I want to configure SCL for 100Khz.I configured using baud rate register.But the SCL is at 50Hz.Main OSC freq is 8Mhz.Could you please help me out in solving this issue.



jliu83 says:

July 28, 2012 at 2:50 pm

@Kishore. Fosc and Fcy is not the same. Please check. Fcy is always half of Fosc, so that's probably where your error is.

-J



killBacon says:

July 30, 2012 at 11:50 am

I see that you are using a Delayu function quite often but didn't see its creation. is this from a standard header file?
Thanks so much, really love the tutorials!
GREAT JOB!!



jliu83 says:



July 31, 2012 at 9:27 am

@killBacon. Please read the entire tutorial. You'll find the answer to all your questions.



Daniel Erwin says:

September 10, 2012 at 1:08 pm

I think you are missing the “i2c_restart()” function in this tutorial. The “send_i2c_byte(data)” function is copied twice.



jliu83 says:

September 11, 2012 at 9:02 am

@Daniel. void i2c_restart(void) looks fine to me. Can you be more specific?

-J



lmac says:

October 5, 2012 at 1:24 am

@ Daniel – I agree. There is no code for void i2c_restar(void)

The code box underneath “void i2c_restart(void)” contains code for the void reset_i2c_bus(void) function.

The next two code boxes (under “void reset_i2c_bus(void)” and under “char send_i2c_byte(int data)”) contain the same function – “char send_i2c_byte(int data)”

Hope this makes sense.

1

P.S. Excellent tutorial. Saved me a lot of time. Thanks very much.



jliu83 *says:*

October 5, 2012 at 4:09 am

@Imac. I'll look into that.

-J



jliu83 *says:*

October 5, 2012 at 4:53 am

@Imac. Fixed. I can't believe I copied that part of the code wrong all these years... Thanks.

-J



baskapteijn *says:*

January 7, 2013 at 6:11 am

Hello jliu83,

Nice tutorial I might say. I was wondering however, why do you insist on clearing the interrupt flag when you don't seem to use interrupts at all (since you use the polling method)?

gr baskapteijn



jliu83 says:

January 7, 2013 at 6:44 am

@baskapteijn. I don't always use the polling method. Some times the code calls for an interrupt to be used, in which case the same code can be reused without modification.

-J



Lance says:

January 23, 2013 at 12:38 am

Excellent tutorial ! Thank you very much! I will using the i2c in my projects soon...

BTW, why not use the 's function (like StartI2C1,IdleI2C1,MasterwriteI2C1,etc.,)instead of writing by yourself?

Thanks.



jliu83 says:

January 23, 2013 at 7:41 am

@Lance. When I wrote that portion of the tutorial, the Microchip library was incomplete. The PIC24 had just came out and the engineers over at Microchip where still trying to complete the libraries. The libraries are now quite robust and should be used to save time.

-J



Lance says:

January 23, 2013 at 12:03 pm

@J

Ok, but your code is really useful for me! thanks ! I will start my i2c test now, do you have any suggestion about the IO port configuration? I just know that the IO should be set as OD output?

Thank you!



jliu83 says:

January 23, 2013 at 3:04 pm

@Lance. Please read over the IO port section of the tutorial. It should cover all the intricacies of how to configure your peripherals. Since you are using a peripheral IO feature, I believe you do not need to set it as an output, only the RPx pins.

-J



Lance says:

January 23, 2013 at 4:00 pm

@J

Yeah I am now reading all of the tutorial. It's really nice than the official documents! Thank you so much for your kindly help.



Lance says:

January 23, 2013 at 5:37 pm

Hi J,

there is not RPx pins in PIC24F32KA302



James says:

February 13, 2013 at 4:44 am

Hi – Your site is very helpful. It is brilliant and easy to read. I was hoping someone here could help me. I am new to PICs as I have used AVR's previously.

I can't seem to get the I2C module to generate a start condition – I am using a PIC24EP512FU810. I have placed pull ups on the I2C pins and see no activity when I use your code above or mine. Any suggestions?

If you have a simple piece of code to setup the PIC and generate a start condition I'd be happy to test it. I am just so very confused.

Thanks!



Rio says:

May 22, 2013 at 11:36 pm

Kudos J for such an epic forum. To be honest, all the tutorials could not be more helpful to me in my final year project. Although I have a few question.

I am currently using a DSPIC30F4011 to communicate with an altimeter MPL3115A2 through I2C. I have checked my circuitry so many times and everything is up to the standard. I am using 4.7K ohm pull up resistors on the I2C bus. I have tested good few codes that were mainly built for I2C communications between Dspics and other sensors that I found online. Finally, I used your code (of course, changed the memory registers to match those of the Dspic30f) . My problem is with the sequence I guess. When I am sending to the

Unfortunately this is my first encounter with I2C so my apologies for any mistakes.



jliu83 says:

May 23, 2013 at 3:33 am

@Rio. I guess something cut off your question. Care to repost?

-J



Rio says:

May 26, 2013 at 2:55 pm

No Worries,

Kudos J for such an epic forum. To be honest, all the tutorials could not be more helpful to me in my final year project. Although I have a few question.

I am currently using a DSPIC30F4011 to communicate with an altimeter MPL3115A2 through I2C. I have checked my circuitry so many times and everything is up to the standard. I am using 4.7K ohm pull up resistors on the I2C bus. I have tested good few codes that were mainly built for I2C communications between Dspics and other sensors that I found online. Finally, I used your code (of course, changed the memory registers to match those of the Dspic30f) . My problem is with the sequence I guess. When I am sending to the

Unfortunately this is my first encounter with I2C so my apologies for any mistakes.



Rio says:

May 27, 2013 at 10:45 am

Crap, I am sorry, I just noticed the unfinished part.

My problem is with the sequence I guess. When I am sending to the sensor a write or a read command I would not get a response. I would see the SDA output from the dsPIC on the oscilloscope but I would not receive an acknowledgement no matter what am sending or which address am using.

For example, the sensor's slave write address is 0xC0 while the read address is 0xC1 and at the same time it has a slave address of 0x60. I used the three addresses either in a polling function similar to yours or with a read function, yet I am not getting a response from the altimeter.

What am I doing wrong?

Note: I have put excessive voltage through the first altimeter by mistake, so I got another one and I am extremely cautious with it, so i doubt that it might be busted too.



Rahul Guiris *says:*

September 5, 2013 at 7:18 am

Hey Guys, I just implementing a code for I2C on PIC24H. I have a systems were there are 5 Master uC and one Slave uC. I am succesful in transmitting to the slave and it works great. However when add simple loop before the start condition

```
while (1){
```

```
while(I2C1STATbits.S); — Wait if start condition is detected last. When this is commented the code works. I have also tried
```

```
while(I2C1STATbits.S == 0);
```

```
I2C_Start();
```

```
I2C_Send(SLAVE|Write);
```

```
I2C_Send(ADCDData);
```

```
I2C_Stop();
```

```
}
```

The programme does not go past this point.

A problem I am facing is that I2C1STATbits.S is always set. I used the idle function you provided on the website and still no luck. I dont understand what I am doing wrong.

The wait is neccesary so to avoid bus collison if another master is active. Can anyone help.
Thanks in advance.



delphes *says:*

October 5, 2013 at 1:03 pm

Dear All,

Where the DelayuSec is defined ?

Thanks & Regards



Justin *says:*

November 1, 2013 at 9:43 am

<http://www.microchip.com/forums/tm.aspx?m=477521>

I used the above code , modified the just pasted it in. I tried implementing as a header file first, but it did not work as planned. I will figure it out later. But the dirty way works, just pasting it in to the code